Honors research · AQFM air quality model

What changes, what stays, and what we cannot change

Our model was built and trained on Delhi only. This document explains, point by point, why that stops it working in another city, what sir's guide fixes, what we still have to fix, what must stay the same, and what cannot be changed. Every point first explains how that part works, so you do not need to remember the details. The numbers come from our real data unless a point says they are made up.

Start here · First decision · A: Guide's fixes · B: Our fixes · C: Keep · D: Cannot change · E: All India · Questions for sir · Order of work

Start here: how the model uses other stations

Everything in this document is about one thing: how the model uses other stations when it forecasts one station. Here is that process for ITO in central Delhi.

1. **Pick a group.** To forecast ITO, the model takes ITO and its 11 nearest stations, which makes a group of 12. ITO is the main station and the other 11 are its neighbours. They run from Major Dhyan Chand Stadium (1.8 km) to North Campus (7.3 km).
2. **Tell the model where each station is.** Each station's location is turned into numbers, so the model can tell the stations apart.
3. **Cut the past into pieces.** The past 128 hours of each station are cut into 4-hour blocks.
4. **Let stations listen to each other.** In each 4-hour block, every station shares out its attention across the 12 stations in the group. The shares add up to 100%. A station that is upwind gets a higher share, because its pollution is coming this way. This extra score is the wind bonus.
5. **Forecast.** The model uses what it gathered to forecast the next 72 hours.

Delhi is a special case: the stations are close together, there are many of them, and the model has seen nothing else. Each point in section A is a place where the model assumes those things.

Main station `anchor`	The station we are forecasting for.
Neighbours	The other stations in its group, up to 11.
4-hour block `patch`	16 readings in a row from one station, for one measurement. The model reads data in these pieces.
Attention	How a station decides how much to listen to each station in its group.
Training	The model learns from 2018–2023 data.
The check `validation`	During training, 2024 data is used to pick the best version of the model.
Final test	2025 data, used once at the end to score the model.

Hidden stations `hold-out`	Stations kept out of training, so we can later test the model on stations it never learned from.
Zero-shot test	Using a Delhi-trained model on Mumbai with no Mumbai training at all.
Fine-tuning	Continuing to train an already trained model on a small amount of new data.
ERA5	A free worldwide weather dataset. It gives weather at points on a grid, once per hour.
Today's model `v1`	The model we have now, with 23.8 million learned numbers.
New model `v2, city preset`	The smaller model in the guide, with 1.17 million learned numbers.
Code update `aqfm_v2_changes.patch`	The file in the Drive folder that holds the guide's code changes. It has not been added to our code yet.

`First`

Decide this first: two cities or all India?

We have been thinking of the goal as all India. The guide says something different. It says "stop at two cities": train on Delhi, test on Mumbai, then fine-tune on Mumbai. It also says to avoid the words "foundation model for India".

The older design was all India first, with about 550 stations and about 150 million learned numbers. So the written plan actually became smaller. The new code would run in any city, but the data, the experiments and the claims are planned for two cities.

Everything below follows the two-city plan. Section E lists the extra work if sir wants all India.

A What was too Delhi-specific, and the guide's fix

All 9 fixes are in the guide's code update.

A1 Delhi's location was built into the model

How it works today

The model has to know where each station is. It takes the station's latitude and longitude, subtracts Delhi's centre (28.6, 77.2), and multiplies by 10. ITO becomes (0.28, 0.41). All Delhi stations become small numbers between about −2 and +2.5. Those are the only location values the model has ever seen.

Why it is a problem

A station in Mumbai becomes about (−95, −43). That is about 39 times bigger than anything seen in training. The model has learned nothing about numbers that big, so its sense of where a station is breaks down. It is like teaching a child only the numbers 1 to 3 and then asking about 95.

What changes

Location is measured from the main station, in km east and km north, then divided by 25. When we forecast ITO, ITO is (0, 0). Chandni Chowk, 0.9 km west and 3.1 km north, is (−0.03, +0.12). In Mumbai the main station is also (0, 0).

What this gives us

The model stops learning "this place is in Delhi" and learns "my neighbour is 3 km north of me". That means the same thing in every city. With the 100 km limit in A2, no value can go above 4.

A2 Always the 12 nearest stations, however far

How it works today

The model always takes the 11 nearest neighbours and never checks how far they are. In Delhi this is safe. The 11th neighbour is usually 8.3 km away, and at most 21.5 km.

Why it is a problem

Where stations are few, the 11 nearest can be very far away. Imagine a town with only 3 other stations nearby. The model would fill the other 8 places with stations that could be hundreds of km away. The air there has nothing to do with this town, so those stations only confuse the forecast.

What changes

Take up to 11 neighbours, but only within 100 km. That town now has 4 real stations and 8 empty slots. The model still keeps 12 slots, because every example must be the same size. Empty slots are marked as empty and ignored everywhere.

In Delhi

Nothing changes, because all 37 stations have their 11 neighbours inside 100 km. This fix only matters in places with fewer stations.

A3 No way to say "nothing here is useful"

How it works today

Each station shares out 100% of its attention across the 12 stations in its group, including itself. So it can listen mostly to itself. But the shares must always add up to 100%, and every choice brings in some information.

Why it is a problem

This hurts most when a station's own reading is missing for a 4-hour block. Its own share is removed, so the full 100% has to go to the other stations, even if they are far away or unhelpful.

What changes

Add one more choice beside the stations: "listen to nobody". It carries no information when training starts. If a station gives it 70%, only 30% of the normal update comes from stations.

Example

Imagine a station whose only neighbour is 90 km away and the wind is blowing the other way. It can now put most of its attention on "listen to nobody" and avoid being misled.

A4 One fixed idea of "near"

How it works today

One of the clues in the wind bonus is a closeness score, $e^{-\text{distance} \div 25}$. A score near 1 means very close and near 0 means far. The 25 km is a number we chose, and the model cannot change it. The model also gets plain distance as a separate clue, so this score is not its only information.

Why it is a problem

Inside Delhi this score hardly changes, so it says little about which neighbour is really closer. You can see this in the "Today" column below. Also, 25 km was picked for Delhi, and another city may need something else.

What changes

Use three closeness scores, starting at 3, 10 and 40 km. The 3 km score separates very close neighbours, and the 40 km score still gives a signal far away. These are starting values, and training can move them to whatever works best.

Distance	Today: 25 km	New: 3 km	New: 10 km	New: 40 km
1.8 km	0.93	0.55	0.84	0.96
7.3 km	0.75	0.09	0.48	0.83
30 km	0.30	0.00	0.05	0.47
100 km	0.02	0.00	0.00	0.08

A5 No direct clue for travel time

How it works today

The model already uses wind. For every pair of stations, in every 4-hour block, it gets 6 clues: distance, two direction clues, wind speed, direction × speed, and closeness. So it knows which neighbours are upwind. In our own test, removing wind direction made the 24-hour Severe-band error 3.4 µg/m³ worse (one seed).

Two things are missing. Travel time is never worked out as a number. And ITO's 20:00–24:00 block looks directly only at other stations' 20:00–24:00 blocks. Older information from a neighbour does reach ITO, but indirectly: the neighbour first reads its own past, and then ITO reads the neighbour.

Why it is a problem

Polluted air takes time to travel. At a wind speed of 2 m/s, air covers 7.2 km in an hour, so 30 km takes about 4.2 hours. What happened 30 km upwind during 16:00–20:00 reaches ITO during 20:00–24:00. In Delhi this shortcut mostly works: across the training years, air from upwind neighbours took a median of 1.3 hours, and 90% of cases took under 4 hours. Where neighbours are farther away, the delay grows and the shortcut works less well.

What changes

There are two additions. The first is a travel-time clue:

```
time = distance × cos(angle) ÷ (3.6 × wind speed)
```

The 3.6 changes m/s into km per hour. cos(angle) is +1 when the neighbour is exactly upwind and −1 when it is exactly downwind. Wind below 0.5 m/s is counted as 0.5, so we never divide by zero.

The second is that each station can also read its neighbours' previous 4-hour block. So the choices go from 12 to 25: 12 stations now, the same 12 one block earlier, and "listen to nobody".

Real example

In the last 4 hours before midnight on 31 Dec 2024, wind at ITO averaged 1.49 m/s. Chandni Chowk is 3.2 km away and directly upwind, so time = 3.2 × 1.00 ÷ (3.6 × 1.49) = 0.60 hours. North Campus, 7.3 km upwind, gives 1.34 hours. A negative answer means the neighbour is downwind, as with Nehru Nagar at −1.25 hours.

A6 Only a rough sense of how crowded the area is

How it works today

Each station's description includes one number for this: how many other stations are within 5 km. ITO has 7. It is worked out once before training and never changes. When stations listen to each other, the model does know each neighbour's distance and skips broken stations, but the part that answers forecast questions does not get this directly.

Why it is a problem

A station with working neighbours 2 km away can lean on them. A station whose neighbours are far away or down cannot. One fixed count cannot show either of these, as the two real examples below show.

What changes

Five numbers go into each station's description, using only stations that have data in that example: distance to the nearest neighbour, average neighbour distance, share of slots within 5 km, share within 20 km, and share of slots with data.

	Today	New
How many numbers	1	5
When worked out	Once, before training	Fresh for every example
Sees past 5 km	No	Yes
Broken stations	Still counted	Not counted

Real example 1

Same number today, but very different stations:

Station	Today	Nearest	Average	Within 20 km
Patparganj	0	5.2 km	6.8 km	11 of 11
Narela	0	7.3 km	17.1 km	7 of 11

Real example 2

Same station and same number today, but two very different weeks at ITO:

128 hours ending	Today	With data	Nearest	Average	Within 5 km	Within 20 km
31 Dec 2025	7	11 of 11	1.8 km	4.4 km	7 of 11	11 of 11
19 Jul 2023	7	4 of 11	3.5 km	4.5 km	3 of 11	4 of 11

Three of ITO's neighbour locations are wrong (see B4), so its numbers will change after that fix.

A7 Delhi never practises having few neighbours

How it works today
In training, the model always sees Delhi's full group of close stations.

Why it is a problem
It never learns what to do when neighbours are few. It is like learning to drive on one familiar road and then being sent to a different city. A place with fewer or differently spread stations would be new to it.

What changes
In 30% of training examples, 30–70% of the neighbours are removed at random. ITO's 11 neighbours might become 5. The removed places become empty slots, as in A2.

Its limit
This can only remove stations. It cannot add far-away ones, so the model still never sees a long link. See D1.

A8 Weather would be scaled differently in each city

How it works today
Weather numbers are rescaled before the model sees them: (value – average) ÷ spread. Delhi's training-years temperature has an average of 24.8 °C and a spread of 8.2 °C, so 30 °C becomes +0.64. These averages are recalculated from whichever data is loaded, and they are not saved with the model.

Why it is a problem
If we load Mumbai, the averages are recalculated from Mumbai's weather. Imagine Mumbai's average is 27 °C with a spread of 3 °C (made-up numbers, to show the idea). Then 30 °C becomes +1.00. The same temperature now looks like a different number, so the Delhi model misreads Mumbai's weather.

What changes
Save Delhi's averages and spreads inside the saved model, and reuse them on Mumbai. Then 30 °C is +0.64 in both cities.

A9 A bigger city would dominate training

How it works today
Each training example starts by picking one random station, and every station has the same chance.

Why it is a problem
With two cities, more stations means more examples. Imagine Mumbai has 25 stations (a made-up number, because we do not have its data yet). Delhi's 37 stations would then get 60% of all examples, and the model would learn mostly Delhi.

What changes
Pick a city first, 50–50, and then pick a station inside that city.

B Not fixed by the guide, so we must do these

B1 The data steps work only for Delhi

How it works today

Several steps prepare the data before training, and each is fixed to Delhi. The ERA5 download covers only a box around Delhi (latitude 28.2–29.0, longitude 76.5–77.5). The location search adds "Delhi, India" to every station name. The results always go to one folder.

Why it is a problem

Mumbai is near latitude 19, far outside that box, so it would get no weather. The search would look for Mumbai's stations in Delhi. If a search fails, the station silently gets Delhi's centre, so a Mumbai station could end up about 1,150 km away with no warning. The new training code also expects one folder per city.

What we must do

Make the city name, weather box and output folder into inputs. Make a failed search stop with an error. Save each city to its own folder: data/delhi and data/mumbai.

B2 There is no Mumbai data

Where we are

The project has data for Delhi only.

Why it is a problem

Every Mumbai experiment in the guide needs this data, so none of them can start. The code also needs both cities on the same 15-minute timeline, and the Mumbai final test needs 2025 data.

What we must do

Collect Mumbai's 15-minute station readings, the location of each station, and ERA5 weather for a Mumbai box. Then run them through the same data steps as Delhi.

B3 Station facts mean different things in each city

How it works today

Each station has 4 fixed facts: its usual PM2.5, its usual PM2.5/PM10 ratio, a height clue from its air pressure, and how many stations are within 5 km. Each fact is rescaled using that city's own average and spread. ITO's usual PM2.5 is about 82 µg/m³ against a Delhi station average of about 71, so its fact comes out as +1.18.

Why it is a problem

+1.18 means "dirtier than the average station in this city". In Mumbai the same +1.18 would mean "dirtier than the average Mumbai station", which may be a very different amount of pollution. So one number has two meanings.

There is a second problem. The first two facts come from the station's own PM2.5 history. A brand-new location has no history, so those facts do not exist there. Even a "hidden" station is not fully hidden, because its history has been fed in through these facts.

What we must do

Use facts that mean the same everywhere, such as site type, elevation or land use, as the guide suggests. Rescale them with one fixed rule for all cities. The "stations within 5 km" fact can go, because A6 replaces it.

B4 Five Delhi stations have doubtful locations

How it works today

Station locations were found by searching their names on OpenStreetMap. Three searches failed: IHBAS, IMD Lodhi Road and NSIT Dwarka. All three were given the same default point at Delhi's centre (28.6139, 77.209). The two Pusa stations (DPCC and IITM) also share one exact point.

Why it is a problem

The model thinks those 3 stations are 0 km apart, and that NSIT Dwarka is 3.5 km from ITO, although Dwarka is in south-west Delhi. The new model uses distance much more than today's model does: for position, crowding and travel time. So wrong locations do more damage. Weather is also taken from each location, so those values are off too.

What we must do

Look up the real locations of the 3 stations, and check whether the 2 Pusa stations really are at one point. Update stations.csv and rebuild the data. Do this for the new model only (see C12).

B5 Hidden stations leak into choosing the best model

How it works today

To test stations the model never learned from, the guide hides 4 Delhi stations from training. During training, the check on 2024 data picks the best version of the model. The guide's code stops hidden stations from being the main station in this check.

Why it is a problem

It does not stop them from being neighbours, and the check scores all 12 stations in each group. So hidden stations still affect which version gets picked. Across 5 random choices of 4 hidden stations, 48–97% of groups contained one, and 7–12% of all station slots were hidden stations. The guide says these scores are clean, but they are not.

What we must do

Give the check the same list of allowed stations that training uses, so hidden stations cannot appear at all. This is a small code change.

B6 The unseen-station test is not virtual sensing

What the guide says

Virtual sensing means estimating air quality at a place with no sensor. The guide labels its experiment E2 this way. E2 hides 4 stations during training and then tests on them.

Why it is a problem

At test time, the code still gives the hidden station its own past 128 hours of readings. A place with no sensor would have none. So E2 really shows "forecasting at a new station that already has a sensor". That is a useful result, but it is a different claim.

What we must do

Choose one. Either rename E2 to say what it really tests, or also hide the station's own readings so it becomes true virtual sensing. The second option is a harder test and needs extra code.

B7 The code update does not fit our current code

Where we are

The guide's changes come as a code update file. It was made from an older copy of our code, before we added options such as use_blh and --pm25-direct.

Why it is a problem

A trial run shows 2 sections fail, one in data.py and one in train.py. Our files also use Windows line endings, which must match the update first.

What we must do

Add it on a new git branch, merge the 2 sections by hand, and keep our options. Then confirm that with all new switches off, the code still gives the same outputs as today's model.

C Should not be changed

These are correct already, or the guide says to keep them. Changing them would waste time or make the results less fair.

C1 The formula that chooses neighbours

What it is

The code finds the distance between stations with the haversine formula, which treats the Earth as a ball.

Why we keep it

It is correct for any two places on Earth, so it already works for Mumbai or anywhere else in India.

C2 The flat-map distance

What it is

A second, simpler formula treats a small area as if it were flat. It is used to work out wind direction between stations, and the new model also uses it for position and crowding. Its code comment mentions Delhi's 60 km, so it looks Delhi-only. This is the "local only" distance calculation.

Why we keep it

It is not tied to Delhi, because it recentres on each group of stations. We measured its worst case across India, with 11 stations close together and 1 far away. The 100 km limit keeps us in the first row, so the guide keeps it and we should too.

Far station	Distance error	Direction error
100 km	0.16 km	0.3°
300 km	1.5 km	0.9°
1,000 km	19 km	3.4°
2,800 km	195 km	11°

C3 The +5:30 time shift

What it is ERA5 weather uses UTC time and station readings use Indian time. We add 5 hours 30 minutes so the two line up.

Why we keep it All of India uses one time zone, so Mumbai and every other Indian city need exactly the same shift.

C4 Wind stored as east and north arrows

What it is Wind speed and direction are turned into an east part and a north part. This avoids problems such as 350° and 10° averaging to 180°, which points the wrong way.

Why we keep it The maths is the same in every city, so nothing about it is specific to Delhi.

C5 The shape of each example

What it is 15-minute readings, 4-hour blocks, 128 hours of past, 72 hours of future, and a group of 12 slots.

Why we keep it The new settings do not change any of these. Keeping them means Delhi and Mumbai examples have the same shape, and the model can be compared at 1, 6, 24 and 72 hours like before.

C6 The core of the model

What it is The model predicts 8 items and rebuilds PM2.5 as PM10 × a fraction, which keeps PM2.5 below PM10. You ask it questions (which item, which station, how far ahead) and it answers them.

Why we keep it The guide names this as our strongest argument: one model forecasts and fills gaps with the same weights. The code update does not touch the physics, loss or masking files.

C7 The years and the scoring

What it is Train on 2018–2023, check on 2024, and test on 2025. Forecast from every midnight, and score at 1, 6, 24 and 72 hours with the same scoring code as the baselines.

Why we keep it If these change, new results can no longer be compared fairly with the old ones or with the baselines.

C8 No city name or station ID inside the model

What it is

The model has no list of city names or station IDs. In the new model it knows a station only through relative position and its station facts. The code update also has an option to add absolute location, and that option stays off.

Why we keep it

A model built on IDs cannot work at a new station, because it has no ID for it. The absolute-location option is meant for many cities. With only two, it would just learn "this is Delhi" or "this is Mumbai".

C9 One look-back block

What it is

In A5, each station reads its neighbours' previous 4-hour block. That is one look-back block.

Why we keep it

At 2 m/s, 30 km takes about one block. The guide says this is enough at city scale, and that 2–3 are only for regional or national neighbours. Each extra block adds 12 more choices for every station.

C10 Delhi's weather scale during Mumbai fine-tuning

What it is

The saved Delhi averages from A8 stay in use while the model is fine-tuned on Mumbai.

Why we keep it

The model learned weather on Delhi's scale, so changing the scale would change what every weather number means. The guide makes this the default, and allows refitting only as a comparison experiment.

C11 Test on Mumbai before any Mumbai training

What it is

This is the zero-shot test: the Delhi-trained model forecasts Mumbai's 2025 data with no Mumbai training.

Why we keep it

The guide calls this the most important number for the claim. If Mumbai data influences the model first, it no longer shows how well a Delhi model moves to a new city.

C12 Today's results and data

What it is

Our 23.8 million model, its results tables and the data behind them.

Why we keep it

They are our finished Delhi results. Fixing locations changes the data, so the old results could no longer be reproduced from it. Keep a copy of the old data, and apply the fixes to the new model only.

D Cannot be changed

These are limits of the data or of the method. We cannot remove them, so we state them clearly and work around them.

D1 Delhi is small

The limit

The two farthest Delhi stations are 42.4 km apart, and the longest link in any group of 12 is 21.5 km. So a model trained only on Delhi never sees a 50 km link. Removing neighbours (A7) does not help, because that only makes groups smaller.

What we can do

Report Mumbai results by distance to the nearest neighbour: 0–5, 5–15 and over 15 km, as the guide asks. The joint Delhi and Mumbai model can see longer links if Mumbai has them. Adding stations from Gurugram, Noida and Faridabad might also help, but that idea is untested.

D2 ERA5 weather is coarse

The limit

ERA5 points are about 25 km apart. All 37 Delhi stations sit inside just 5 of its grid squares, so many stations get nearly the same weather even when local conditions differ. This may matter more in a coastal city like Mumbai, where one square can cover both land and sea.

What we can do

We cannot change the grid. We keep blending the 4 nearest grid points, and we keep using a station's own weather readings wherever they exist.

D3 We cannot create stations that do not exist

The limit

The spatial part of the model learns from real neighbours. If a station has none within 100 km, that part adds nothing, and the forecast relies on the station's own history and the weather.

What we can do

Report sparse stations separately. The guide says the model with neighbours should never do worse than a model with no neighbours in sparse areas. If it does, then A2 or A3 is not working.

D4 The old trained model cannot be reused

The limit

The new model has a different size. Today's model uses 384 numbers per vector and 23.8 million learned numbers. The new one uses 128 and 1.17 million. The learned numbers of one do not fit into the other, and location also works differently.

What we can do

Train the new model from zero on Delhi. Keep today's model as our Delhi reference result.

D5 Travel time stays an estimate

The limit The formula in A5 uses the wind at one station and assumes air moves in a straight line. Real wind changes along the way. The model also reads data in 4-hour blocks, so it cannot place the arrival of pollution at an exact hour.

What we can do Treat it as a clue and never as a measured fact. The model learns how much to trust it.

D6 Two cities cannot prove all India

The limit A foundation model usually means training across many regions and then adapting to many settings. One source city and one target city is a transfer study. Reviewers may also say that any result comes from this one pair of cities, since Delhi and Mumbai have very different weather.

What we can do Use the guide's title "Towards an urban air-quality foundation model" if the tests go well. Testing on 1–2 more cities without training them is cheap, and it makes the claim stronger.

E Extra work only if sir chooses all India

- **Data for every city.** All of B1 and B2 repeat for each city. The older design estimated about 550 stations across India.
- **Absolute location.** With many cities, place can carry real meaning, such as hills against coast. The option is already in the code update and is off by default. C8 applies only to the two-city plan.
- **More look-back blocks.** At 2 m/s, 100 km takes 13.9 hours, which is 3.5 blocks, so one block cannot reach that far. The guide says to use 2–3 for regional or national neighbours.
- **A new pressure rule.** Stations report pressure in mmHg (about 740 in Delhi) or hPa (about 980). The code treats any reading above 900 as hPa and converts it. That works in Delhi. Above about 990 m, true pressure is below 900 hPa, so a hill station's reading would stay unconverted and be wrong. This comes from a standard calculation and has not been tested on real hill stations.
- **Model size.** The guide chose about 1–3 million learned numbers for the amount of data in two cities. More cities bring more data, so this must be decided again.
- **The claim.** The guide says the plain name "foundation model" only fits with training on many cities and gains across most of them.

Questions for sir

1. **Two cities or all India?** This decides how much data we collect, how big the model is, and what we can claim.
2. **Where will Mumbai's data come from, and does it cover 2018–2025?** The final test needs 2025, and fine-tuning needs months of data before that.
3. **Which 4 station facts should replace the current ones?** They must exist for any location and mean the same in every city (B3).
4. **Should the unseen-station test cover a place with no sensor?** This decides whether we can claim virtual sensing (B6).

Order of work after sir agrees

1. **Fix the station locations (B4).** This comes first because the data for the new model is built from these locations.
2. **Make the data steps work for any city, and collect Mumbai's data (B1, B2).** Nothing about Mumbai can run before this.
3. **Replace the station facts (B3).** The zero-shot test is not fair until the facts mean the same in both cities.
4. **Add the code update on a new branch and merge the 2 failed sections (B7).**
5. **Fix the hidden-station leak (B5).** Otherwise the hidden-station scores are not clean.
6. **Do a short Delhi run.** This confirms that nothing broke before we spend time on full training.
7. **Run the experiments in the guide's order.** Train on Delhi. Test on Mumbai with no Mumbai training. Fine-tune on Mumbai with 1, 3, 6 and 12 months and all data. Train Mumbai from scratch with the same amounts. Then train on both cities together.

How this was checked

- **Measured on our real 37 Delhi stations, using our current code:** all Delhi distances, the ITO examples, the travel times, the flat-map error, the hidden-station leak and the weather averages.
- **Read in full but not run:** the guide's code update. A trial run shows where it fails to fit, but the guide's own test claims are not confirmed yet.
- **Calculated only:** Mumbai's (−95, −43), the 990 m pressure limit and the 100 km travel time.
- **Made up to show an idea, and marked where used:** the town in A2, the station in A3, and Mumbai's temperature and station count in A8 and A9.
- **Not available yet:** Mumbai data, so no Mumbai results exist.